HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology

# FINAL PROJECT REPORT

Course: Introduction to Software Engineering

# Virtual Stock Market Simulation and Consultancy Platform

## System name: SoictStock

Group: *To be filled*
Class code: *To be filled*
Instructor: *To be filled*

Hanoi, June 14, 2026

# Member Assignment

This section records the planned division of work for the project team. The roles should be updated with the actual names and student IDs before submission.

Table 1: Member assignment

| **Full name** | **Student ID** | **Main work** |
|---|---|---|
| Member 1 | To be filled | Team lead, requirement analysis, project planning, report integration, final presentation. |
| Member 2 | To be filled | Backend API, MongoDB connection, authentication, portfolio and trading logic. |
| Member 3 | To be filled | Frontend interface, simulation dashboard, portfolio page, responsive layout. |
| Member 4 | To be filled | Market simulation engine, WebSocket price streaming, scenarios, stock data and order book. |
| Member 5 | To be filled | Learning center, quizzes, badges, chatbot, advisor content, blog and community module. |
| Member 6 | To be filled | Testing, deployment, Docker setup, user guide, screenshots and demo validation. |

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Problem Survey

## 1.1 Problem Description

The proposed project, SoictStock, is a web-based virtual stock market simulation and financial education platform. The system is designed for users who want to understand how equity trading works without using real money. Many beginners find concepts such as price movement, market volatility, order types, portfolio allocation, profit and loss, drawdown, and risk management difficult to understand through theory alone. Real trading platforms are not suitable for early learning because they expose inexperienced users to financial risk.

SoictStock addresses this problem by providing a risk-free environment where users can observe simulated stock prices, place buy and sell orders, manage a virtual portfolio, join trading contests, learn financial concepts through lessons and quizzes, and ask a chatbot for educational guidance. The system also introduces SoictStock's consultancy direction through advisor profiles, educational blog posts, market insights, and a mock advisor/backtesting service.

The system is strictly intended for education and simulation. It does not support real-money trading, brokerage integration, or legally binding financial advice. All transactions, market prices, consultant profiles, rewards, and advisory responses are simulated.

## 1.2 Problem Survey and Existing Practices

Current financial learning methods often fall into three groups. First, traditional lessons and articles explain stock market concepts but provide limited interaction. Second, real trading platforms provide practical experience but require real money and may lead to losses. Third, simple stock market games allow simulated trading but often lack structured learning, realistic market behavior, portfolio analytics, and explanation of

risk.

The target users of SoictStock need an integrated learning environment that combines:

- real-time simulated stock prices;
- virtual buy/sell trading and order practice;
- portfolio analytics and risk metrics;
- lessons, quizzes, badges, and practice tasks;
- chatbot support for financial concepts and platform usage;
- blogs and community discussion;
- contests and leaderboards to increase engagement.

Therefore, the project is not only a trading simulator. It is an educational ecosystem that connects market simulation, learning, portfolio reflection, and community engagement.

## 1.3 Client, Users, and Stakeholders

Table 1.1: Client, users, and stakeholders

| Role | Description |
|---|---|
| Client / Course evaluator | The instructor and teaching team who define the course requirements and evaluate the final software engineering report and demo. |
| Learner / Registered user | A user who signs up, receives virtual cash, trades simulated stocks, studies lessons, completes quizzes, and uses the chatbot. |
| Competitive user | A user who joins contests, trades with a contest portfolio, and competes on the leaderboard. |
| Blog author | A signed-in user who creates draft posts, publishes educational posts, archives posts, and interacts with comments and votes. |
| Platform maintainer | A developer or operator who maintains stock data, learning content, contests, deployment configuration, and service availability. |
| SoictStock consultant profile | A simulated professional profile used to present consultancy-oriented educational content. |
| Development team | The group responsible for requirements, design, implementation, testing, deployment, and documentation. |
| Stakeholders | Learners, instructors, the development team, and people interested in financial technology education. |

## 1.4 Project Scope

### 1.4.1 In Scope

The implemented project scope includes:

- user sign-up, sign-in, and profile update;
- virtual wallet and default portfolio initialization;
- simulated stock list, quotes, and historical price charts;
- real-time price updates through WebSocket;
- buy/sell simulated trades and order management;
- portfolio value, holdings, realized and unrealized profit/loss;
- risk metrics such as volatility, Sharpe ratio, and maximum drawdown;

- leaderboard and trading contests;
- market scenarios such as crisis, technology bubble, pandemic shock, and inflation scenario;
- news display and news injection for market context;
- learning paths, lessons, quizzes, badges, market analysis lab, and pattern game;
- chatbot learning assistant and mock advisor/backtesting service;
- blog/community module with drafts, publishing, archiving, votes, and comments;
- Docker-based deployment support.

### 1.4.2 Out of Scope

The following features are not included in the current project version:

- real-money trading;
- real brokerage or stock exchange integration;
- legal financial advice;
- real consultant booking and payment;
- production-grade role-based admin dashboard;
- production-grade authentication session management;
- real contest rewards.

## 1.5 Basic Business Processes

This section summarizes the main business processes of SoictStock using an input–process–output structure.

Table 1.2: Business Process 1: User registration and portfolio initialization

| **Input** | **Process** | **Output** |
|---|---|---|
| Email, password, display name | Validate account data, hash the password, create a user record, and initialize a default portfolio with virtual cash. | User account, hashed password, default portfolio, initial virtual cash balance. |

Table 1.3: Business Process 2: Real-time market simulation

| **Input** | **Process** | **Output** |
|---|---|---|

| | | |
|---|---|---|
| Stock list, base price, sector, drift, volatility, market regime | Generate historical bars and real-time ticks using the simulation engine. Apply jump events, momentum, mean reversion, volatility clustering, scenarios, and news impact. | Current prices, stock quotes, historical bars, real-time WebSocket price stream. |

Table 1.4: Business Process 3: Virtual trading

| **Input** | **Process** | **Output** |
|---|---|---|
| User ID, ticker, trade type, quantity, current price | Validate ticker, check virtual cash or holdings, execute buy/sell logic, update portfolio, record transaction, and update leaderboard. | Filled transaction, updated cash, updated holdings, updated leaderboard entry. |

Table 1.5: Business Process 4: Portfolio and risk analytics

| **Input** | **Process** | **Output** |
|---|---|---|
| Holdings, cash, transactions, price stream, portfolio snapshots | Calculate total value, realized/unrealized profit and loss, allocation, volatility, Sharpe ratio, and maximum drawdown. | Portfolio dashboard, allocation chart, transaction history, risk metrics. |

Table 1.6: Business Process 5: Learning and quiz progress

| **Input** | **Process** | **Output** |
|---|---|---|
| Lesson ID, section index, quiz answers, user ID | Mark completed sections, calculate quiz score, save quiz attempt, update progress and badges, and recommend next activities. | Learning progress, quiz result, badges, recommended lesson or task. |

Table 1.7: Business Process 6: Chatbot and advisor support

| Input | Process | Output |
|---|---|---|
| User message, market context, portfolio context, learning context | Detect intent, apply safety rules, generate an educational answer, suggest relevant pages or lessons, and store chat history. | Chatbot response, suggestion cards, educational disclaimer, saved history. |

Table 1.8: Business Process 7: Blog and community discussion

| Input | Process | Output |
|---|---|---|
| Blog title, content, stock tag, author ID | Create draft, update content, publish, archive, delete, vote, and comment on posts. | Published post, archived post, public profile posts, comments, votes. |

Table 1.9: Business Process 8: Contest and leaderboard

| Input | Process | Output |
|---|---|---|
| Contest ID, user ID, contest trade | Join active contest, initialize contest portfolio, restrict allowed tickers, execute contest trade, and rank participants. | Contest portfolio, contest transaction, contest leaderboard, reward/ranking display. |

## 1.6 Business Function Decomposition

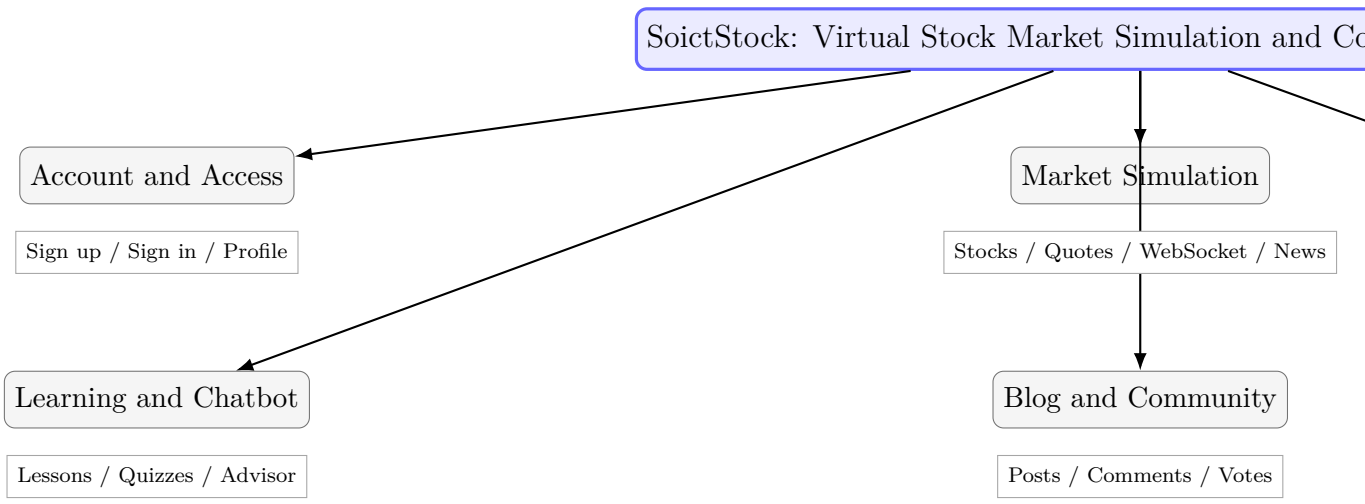


Figure 1.1: Business function decomposition of SoictStock

Table 1.10: Business function decomposition table

| No. | Business function | Description | Status |
|---|---|---|---|
| 1 | Account and access management | Manage sign-up, sign-in, profile update, and default portfolio initialization. | Implemented |
| 2 | Market simulation management | Generate historical bars, real-time ticks, quotes, scenarios, and news-driven context. | Implemented |
| 3 | Trading management | Support direct buy/sell transactions, open orders, filled orders, and cancellation. | Implemented |
| 4 | Portfolio analytics | Calculate holdings, cash, portfolio value, profit/loss, allocation, volatility, Sharpe ratio, and maximum drawdown. | Implemented |
| 5 | Learning management | Manage lessons, quizzes, section completion, badges, learning progress, pattern game, and market analysis lab. | Implemented |
| 6 | Chatbot and advisor support | Provide educational Q&A, suggestion cards, safety disclaimers, mock advisor responses, and mock backtesting. | Implemented |
| 7 | Blog and community management | Manage draft posts, published posts, archived posts, comments, votes, and public user profiles. | Implemented |
| 8 | Contest and leaderboard management | Manage active contests, contest portfolios, contest trades, rewards, and rankings. | Implemented |
| 9 | Administration dashboard | A dedicated role-based admin dashboard for all platform data. | Future work |

# Chapter 2

# Software Requirements Specification

## 2.1 General Introduction

This chapter specifies the requirements of SoictStock from the user's and system's perspective. The SRS defines the main actors, use cases, functional requirements, and non-functional requirements. It also provides a basis for system design and testing.

### 2.1.1 Purpose

The purpose of SoictStock is to help users learn stock market concepts through a realistic but risk-free simulation. The system combines trading practice, portfolio tracking, educational content, chatbot support, community discussion, and competitive learning.

### 2.1.2 Definitions

Table 2.1: Glossary

| Term | Definition |
|---|---|
| Virtual stock | A simulated tradable asset representing a fictional or educational stock. |
| Virtual wallet | A simulated cash balance used for educational trading. |
| Portfolio | A collection of virtual cash, stock holdings, transactions, and performance metrics. |
| Order | A request to buy or sell a simulated stock. |
| Transaction | A completed trade that updates portfolio cash and holdings. |
| Tick | A generated price update in the simulated market. |
| Market scenario | A predefined market condition such as crisis or inflation that changes price behavior. |
| Learning path | A structured set of lessons, quizzes, and practice tasks. |
| Advisor | A simulated educational assistant, not a real financial advisor. |

## 2.2 System Actors

Table 2.2: System actors

| Actor | Description |
|---|---|
| Guest | A visitor who can view landing content, public blog posts, and sign-up/sign-in options. |
| Registered user | A user who can trade simulated stocks, view portfolio, learn lessons, take quizzes, create blogs, join contests, and ask the chatbot. |
| Contest participant | A registered user who joins a contest and trades with a contest-specific portfolio. |
| Blog author | A signed-in user who creates, updates, publishes, archives, or deletes their own blog posts. |
| Platform maintainer | A person who maintains the source code, seed data, deployment configuration, and future admin functions. |
| Simulation engine | An internal system component that generates historical prices and real-time ticks. |

| | |
|---|---|
| News injector | An internal service that injects market news into the simulation context. |
| Chatbot service | An internal service that generates educational responses and suggestion cards based on user messages and safety rules. |

## 2.3 Main Use Cases

### 2.3.1 Guest Use Cases

- View landing page.
- Browse public blog posts.
- Sign up.
- Sign in.

### 2.3.2 Registered User Use Cases

- View market dashboard.
- View stock chart and quote.
- Place an order.
- Execute buy/sell trade.
- Cancel an open order.
- View portfolio, risk metrics, and transaction history.
- View leaderboard.
- Join contest and trade in contest arena.
- Read lessons, complete sections, and take quizzes.
- Use market analysis lab and pattern game.
- Ask chatbot.
- Create, update, publish, archive, and delete own blog posts.
- Vote and comment on published blog posts.

### 2.3.3 Simulation Engine Use Cases

- Generate historical price bars.
- Broadcast real-time price ticks.
- Apply market regimes and scenarios.
- Update current quotes.

## 2.4 Use Case Diagram

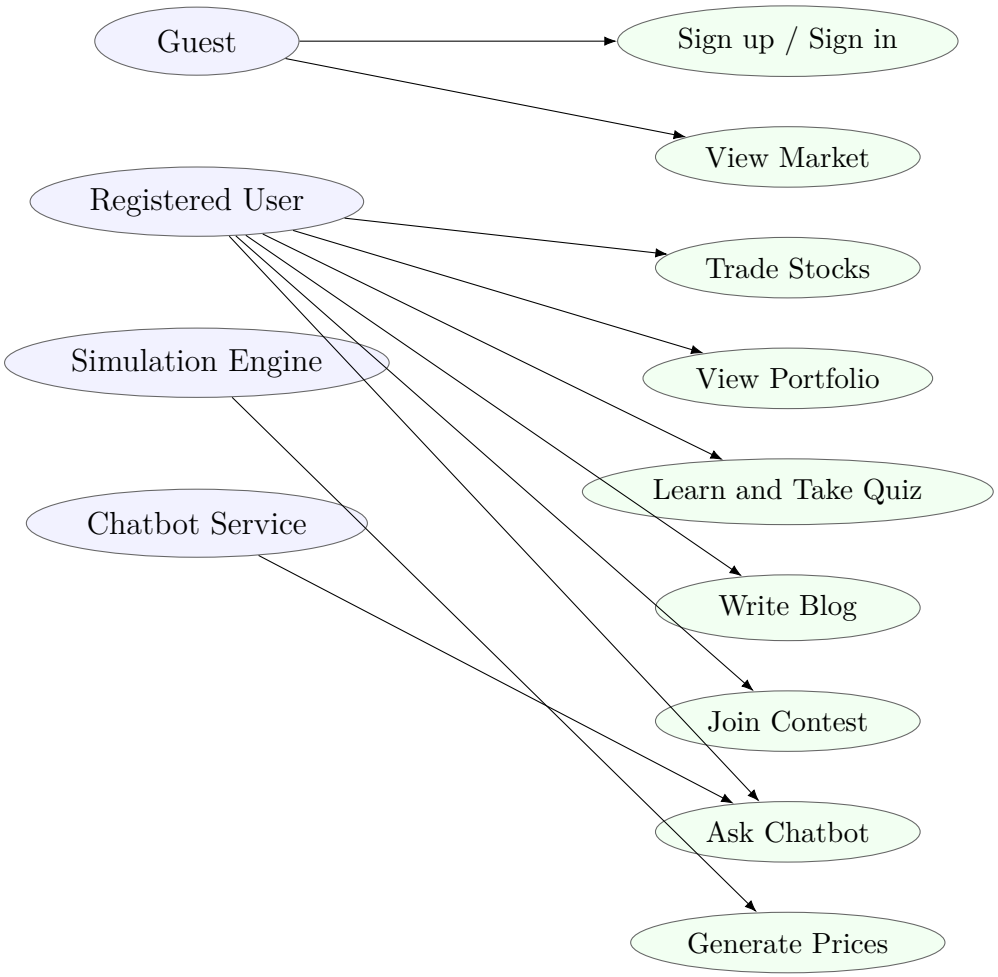


Figure 2.1: Overall use case diagram

## 2.5 Use Case Specifications

### 2.5.1 UC-01: Sign Up

| | |
|---|---|
| Actor | Guest |
| Precondition | The visitor has not signed in. |
| Main flow | The guest opens the authentication modal, enters email, password, and display name. The backend validates the input, hashes the password, creates a user record, and initializes a portfolio with virtual cash. |
| Alternative flow | If the email already exists or the password is invalid, the system returns an error message. |
| Postcondition | A new account and default portfolio are created. |

### 2.5.2 UC-02: Execute Buy/Sell Trade

| | |
|---|---|
| Actor | Registered user |
| Precondition | The user is signed in and the selected ticker exists in the simulated market. |
| Main flow | The user selects a ticker, chooses buy or sell, enters quantity, and submits the trade. The backend reads the current simulated price, validates cash or holdings, updates portfolio, records the transaction, and updates the leaderboard. |
| Alternative flow | If cash is insufficient, holdings are insufficient, or the ticker is unknown, the trade is rejected. |
| Postcondition | Portfolio, transaction history, and leaderboard are updated after a successful trade. |

### 2.5.3 UC-03: Stream Real-time Prices

| | |
|---|---|
| Actor | Simulation engine |
| Precondition | Backend server and WebSocket server are running. |
| Main flow | The engine initializes historical prices, starts a tick loop, updates prices every few seconds, stores ticks, and broadcasts updates to connected frontend clients. |
| Alternative flow | If the WebSocket connection is unavailable, the frontend may fall back to local simulation to keep the user interface responsive. |
| Postcondition | The market dashboard receives current prices without refreshing the page. |

### 2.5.4 UC-04: Complete Quiz

| | |
|---|---|
| Actor | Registered user |
| Precondition | The user opens a lesson or quiz in the learning center. |
| Main flow | The user answers quiz questions and submits the quiz. The system calculates the score, saves the quiz result, updates progress, and displays feedback. |
| Alternative flow | If the user is not signed in, progress may not be persisted. |
| Postcondition | Learning progress and quiz result are stored for the user. |

### 2.5.5 UC-05: Publish Blog Post

| | |
|---|---|
| Actor | Blog author |
| Precondition | The user is signed in and has created a draft post. |
| Main flow | The author edits the title and content, saves the draft, and publishes the post. The system changes the post status to published and makes it visible on the public blog page. |
| Alternative flow | If the user is not the owner of the post, the update, archive, publish, or delete request is rejected. |
| Postcondition | The post is visible to public readers if it is published. |

### 2.5.6 UC-06: Ask Chatbot

| | |
|---|---|
| Actor | Registered user |
| Precondition | The chatbot widget is available. |
| Main flow | The user enters a question. The chatbot service detects intent, applies safety rules, uses platform and learning context, returns an educational response, and saves the message history. |
| Alternative flow | If the user asks for real investment advice, the chatbot responds with an educational disclaimer and avoids a direct buy/sell recommendation. |
| Postcondition | The user receives guidance, and the chat history is updated. |

## 2.6 Functional Requirements

### 2.6.1 FR-01 Authentication and User Profile

FR-AUTH-1 The system shall allow users to sign up using email, password, and display name.
FR-AUTH-2 The system shall hash passwords before storage.
FR-AUTH-3 The system shall allow users to sign in.
FR-AUTH-4 The system shall allow users to view and update their display name.
FR-AUTH-5 The system shall create a default portfolio for each new user.

### 2.6.2 FR-02 Market Simulation

FR-MKT-1 The system shall maintain a list of simulated stocks.
FR-MKT-2 The system shall generate historical price bars for each stock.
FR-MKT-3 The system shall stream real-time price updates through WebSocket.

FR-MKT-4 The system shall provide stock quotes and historical data through REST APIs.

FR-MKT-5 The system shall support market scenarios and regimes.

### 2.6.3 FR-03 Trading and Order Management

FR-TRD-1 The system shall allow users to execute buy and sell trades.

FR-TRD-2 The system shall reject buy trades when virtual cash is insufficient.

FR-TRD-3 The system shall reject sell trades when holdings are insufficient.

FR-TRD-4 The system shall record each successful transaction.

FR-TRD-5 The system shall allow users to place, view, and cancel simulated orders.

### 2.6.4 FR-04 Portfolio Management and Risk Analytics

FR-PRT-1 The system shall display cash balance and current holdings.

FR-PRT-2 The system shall calculate total portfolio value.

FR-PRT-3 The system shall calculate realized and unrealized profit/loss.

FR-PRT-4 The system shall display transaction history.

FR-PRT-5 The system shall calculate risk metrics such as volatility, Sharpe ratio, and maximum drawdown.

### 2.6.5 FR-05 Learning Center

FR-LRN-1 The system shall display learning paths and lessons.

FR-LRN-2 The system shall track completed lesson sections.

FR-LRN-3 The system shall allow users to submit quizzes.

FR-LRN-4 The system shall save quiz results.

FR-LRN-5 The system shall display badges, practice tasks, pattern game, and market analysis lab activities.

### 2.6.6 FR-06 Chatbot, Advisor, Blog, Contest, and Leaderboard

FR-SOC-1 The system shall provide a chatbot widget for educational questions and platform guidance.

FR-SOC-2 The system shall save and clear chatbot history.

FR-SOC-3 The system shall display advisor-oriented educational responses and mock backtest output.

FR-SOC-4 The system shall allow signed-in users to create, publish, archive, and delete their own blog posts.

FR-SOC-5 The system shall allow users to vote and comment on published posts.

FR-SOC-6 The system shall display active contests and allow users to join them.
FR-SOC-7 The system shall maintain general and contest leaderboards.

## 2.7 Non-functional Requirements

Table 2.9: Non-functional requirements

| **Category** | **Requirement** |
|---|---|
| Performance | The frontend should update displayed prices smoothly with the simulated tick interval. |
| Data integrity | Portfolio cash and holdings must not become negative after trades. |
| Reliability | If the WebSocket connection is unavailable, the frontend should continue displaying simulated movement or recover gracefully. |
| Security | Passwords must be hashed before storage. |
| Authorization | Blog editing, archiving, and deleting should be limited to the owner of the post. |
| Usability | Users should be able to navigate to Simulation, Portfolio, Learn, Blogs, Contest, and Leaderboard from the main navigation. |
| Maintainability | Simulation, order, portfolio, learning, chatbot, blog, contest, and leaderboard logic should be separated into routes, services, stores, and components. |
| Scalability | The system should allow adding more stocks, lessons, contests, and blog posts. |
| Portability | The system should run through Docker Compose or manual npm setup. |
| Ethical safety | Chatbot and advisor responses must clarify that the system is for educational simulation and not real financial advice. |

# Chapter 3

# Requirement Analysis

## 3.1 Identifying Analysis Classes

The main analysis classes are identified from the implemented source code modules and the functional requirements. They are grouped by business domain.

Table 3.1: Analysis classes

| **Class group** | **Classes** |
|---|---|
| User and authentication | User, AuthStore, SessionContext, Profile |
| Portfolio and trading | Portfolio, Holding, Order, OrderBook, Transaction, PortfolioSnapshot |
| Market simulation | Stock, Quote, Tick, OHLCVBar, SimulationEngine, MarketScenario, NewsItem |
| Risk and analytics | RiskMetrics, LeaderboardEntry, PortfolioPerformance |
| Learning | LearningPath, Lesson, LessonSection, Quiz, Question, QuizResult, AchievementBadge, PracticeTask, PatternGame |
| Chatbot and advisor | ChatMessage, ChatbotIntent, ChatbotHistory, SuggestionCard, ConsultantProfile, AdvisorResponse |
| Blog and community | BlogPost, Comment, Vote, PublicProfile |
| Contest | Contest, ContestPortfolio, ContestTrade, Reward, ContestLeaderboardEntry |

## 3.2 Main Analysis Scenarios

The following scenarios connect user goals with system behavior and later design diagrams.

Scenario 1: A new user signs up and receives a default portfolio with virtual cash.

Scenario 2: A user opens the simulation dashboard and receives real-time price updates.

Scenario 3: A user buys a stock and the portfolio is updated.

Scenario 4: A user sells a stock and realized profit/loss is updated.

Scenario 5: A user places a limit or stop order and the order is filled or cancelled based on price conditions.

Scenario 6: A market scenario is activated and users observe the effect on simulated prices.

Scenario 7: A user views portfolio analytics and risk metrics.

Scenario 8: A user joins a contest and trades with a contest-specific portfolio.

Scenario 9: A user reads a lesson, completes sections, and submits a quiz.

Scenario 10: A user asks the chatbot for an explanation about risk, order types, or platform usage.

Scenario 11: A user publishes an educational blog post and receives comments and votes.

## 3.3 Sequence Diagrams

The following diagrams are recommended for the final report. Figure 3.1 provides a simplified sequence diagram for trade execution.

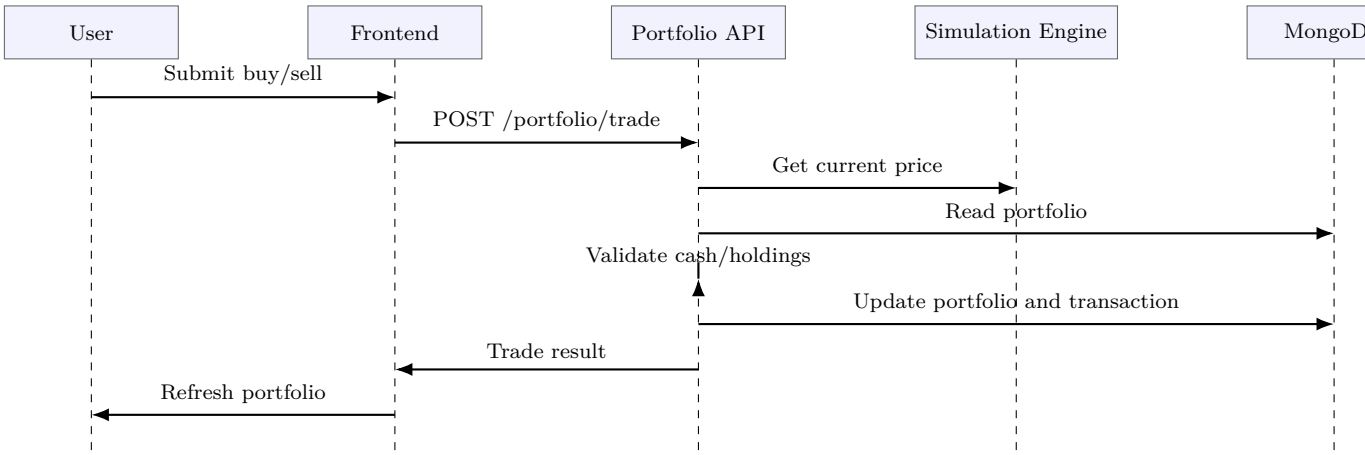


Figure 3.1: Sequence diagram for executing a simulated trade

The final document should also include the following sequence diagrams:

- User sign-up: Guest, Auth Modal, Auth API, MongoDB users, MongoDB portfolios, Auth Store, UI.
- Real-time market streaming: Server, Simulation Engine, MongoDB ticks, WebSocket, Market Store, Simulation Page.
- Place and cancel order: User, Order Form, Orders API, OrderBook Service, database or in-memory order store, Open Orders Panel.
- Complete quiz: User, Quiz Module, Learning Store, Learning API, learning progress collection, Learn Page.
- Chatbot message: User, Chatbot Widget, Chatbot API, Chatbot Service, intent/safety/context modules, chatbot history collection.

## 3.4 Analysis Class Diagram

Figure 3.2 shows the key analysis classes and their relationships.

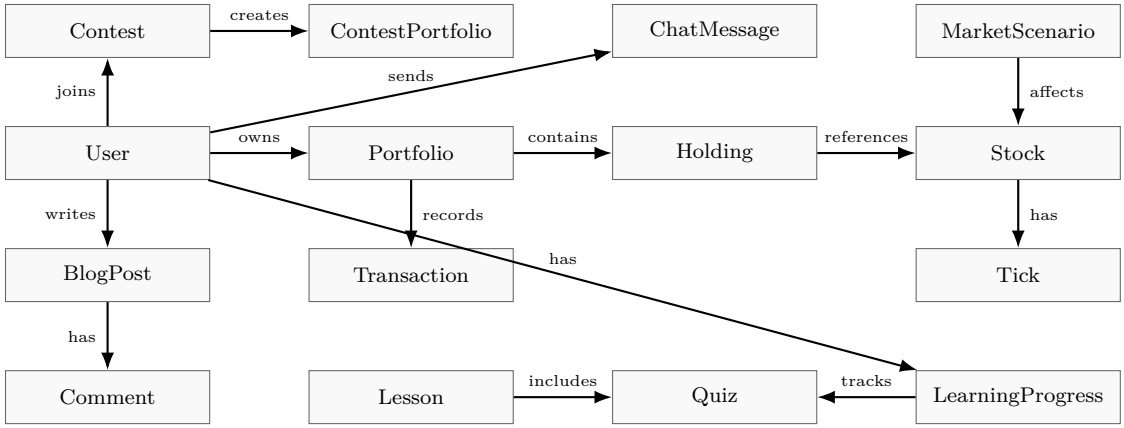


Figure 3.2: Simplified analysis class diagram

## 3.5 Entity Relationship Diagram

The ERD should be drawn around the following data groups:

- User–Portfolio–Transaction: `users`, `portfolios`, `transactions`, `portfolio_snapshots`.
- Market: `stocks`, `ticks`, `news`, `scenarios`.
- Learning: `learning_progress`, lesson data, quiz data, badges.
- Blog: `blog_posts`, comments, votes.
- Contest: `contests`, `contest_portfolios`, contest transactions, contest leaderboard entries.
- Chatbot: `chatbot_history`.

## 3.6 Data Dictionary

### 3.6.1 Collection: users

| Field | Type | Description |
|---|---|---|
| `_id` | ObjectId | User identifier. |
| `email` | String | Login email. |
| `passwordHash` | String | Hashed password. |
| `display_name` | String | User display name. |
| `createdAt` | DateTime | Account creation time. |
| `updatedAt` | DateTime | Last profile update time. |
| `lastLoginAt` | DateTime | Last successful login. |

### 3.6.2 Collection: portfolios

| Field | Type | Description |
|---|---|---|
| `userId` | ObjectId/String | Portfolio owner. |
| `cash` | Number | Current virtual cash. |
| `initialCash` | Number | Initial virtual cash. |
| `holdings` | Object/Array | Ticker, shares, average price, realized profit/loss. |
| `updatedAt` | DateTime | Last portfolio update. |

### 3.6.3 Collection: transactions

| Field | Type | Description |
|---|---|---|
| `userId` | ObjectId/String | Transaction owner. |
| `type` | Enum | Buy or sell. |
| `ticker` | String | Traded stock. |
| `orderType` | String | Market, limit, or stop-loss if applicable. |
| `quantity` | Number | Number of shares. |
| `price` | Number | Executed price. |
| `total` | Number | Total transaction value. |
| `status` | String | Usually filled for completed trades. |
| `createdAt` | DateTime | Transaction time. |

### 3.6.4 Collection: ticks

| Field | Type | Description |
|---|---|---|
| `ticker` | String | Stock ticker. |
| `time` | DateTime | Bar or tick timestamp. |
| `price` | Number | Simulated price. |
| `volume` | Number | Simulated trading volume. |

### 3.6.5 Collection: learning_progress

| Field | Type | Description |
|---|---|---|
| `userId` | ObjectId/String | Learner. |
| `lessonProgress` | Object | Completed sections and current lesson state. |
| `quizResults` | Object/Array | Attempts, score, best score, and pass/fail status. |
| `earnedBadges` | Array | Badges earned by the user. |
| `currentLevel` | String | Learning level. |
| `recommendedLessonId` | String | Recommended next lesson. |

### 3.6.6 Collection: blog_posts

| Field | Type | Description |
|---|---|---|
| `id/slug` | String | Blog identifier. |
| `title` | String | Blog title. |
| `excerpt` | String | Optional short summary. |
| `content` | String | Blog body. |
| `author_id` | String | Author identifier. |
| `status` | Enum | Draft, published, or archived. |
| `stock_tags` | Array | Stock symbols extracted from `$TICKER` mentions. |
| `upvotes/downvotes/ranking` | Number | Voting counters and derived score. |
| `votes_by_user` | Object | Tracks each user's current vote. |
| `comments` | Array | Comments attached to the post. |
| `comment_count` | Number | Number of comments for sorting and display. |

| | | |
|---|---|---|
| `published_at` | DateTime | Publication time. |
| `created_at/updated_at` | DateTime | Audit timestamps. |

# Chapter 4

# System Design

## 4.1 Selected Architecture

SoictStock uses a client–server architecture. The frontend is a React/Vite single-page application. The backend is an Express.js server connected to MongoDB. Real-time stock prices are delivered through a WebSocket endpoint. The backend is organized into route modules and service modules. The frontend is organized into pages, reusable components, Zustand stores, and API helper functions.

The design follows core software engineering principles: modularity, separation of concerns, traceability from requirements to design components, and maintainability. The simulation engine is separated from the user interface and trading logic. The learning, blog, chatbot, contest, and portfolio modules are also separated into dedicated services or stores.

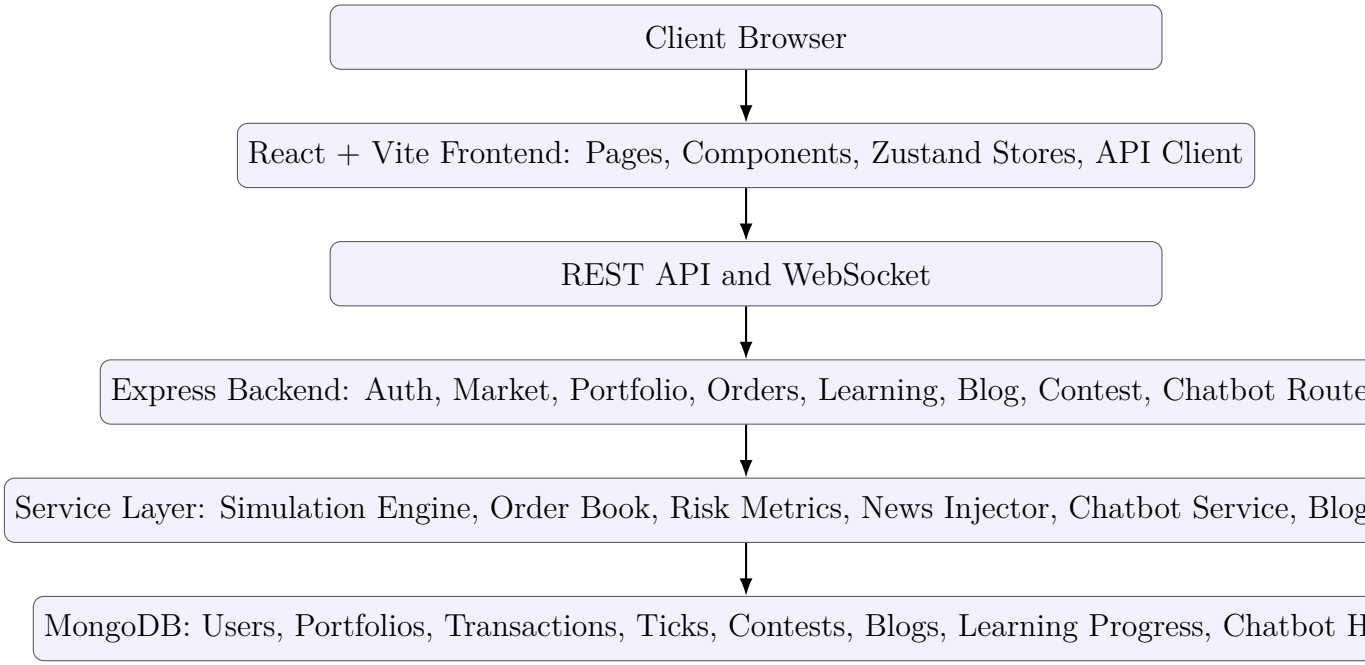


Figure 4.1: High-level architecture of SoictStock

## 4.2 Main Components

Table 4.1: Main components and source mapping

| Component | Responsibility | Source mapping |
|---|---|---|
| Authentication component | Sign-up, sign-in, profile update, default portfolio initialization. | `routes/auth.js`, `authStore.js`, `AuthModal.jsx` |
| Market simulation component | Stock data, quotes, historical prices, real-time ticks, scenarios. | `simulationEngine.js`, `market.js`, `priceStream.js`, `marketStore.js` |
| Trading component | Direct buy/sell trades, order placement, open/filled orders, cancellation. | `orders.js`, `portfolio.js`, `orderBook.js`, `OrderForm.jsx` |
| Portfolio component | Cash, holdings, portfolio value, profit/loss, snapshots, risk metrics. | `portfolio.js`, `riskMetrics.js`, `Portfolio.jsx`, `portfolioStore.js` |
| Learning component | Lessons, quizzes, progress, badges, practice tasks, market lab, pattern game. | `learning.js`, `learningData.js`, `Learn.jsx`, `QuizModule.jsx` |
| Chatbot component | Educational Q&A, context-aware suggestions, safety disclaimers, chat history. | `chatbot.js`, `chatbotService.js`, `ChatbotWidget.jsx` |
| Blog component | Drafts, published posts, archives, comments, votes, stock mentions, filters, and user profiles. | `blogs.js`, `blogService.js`, `Blogs.jsx`, `BlogDetail.jsx`, `BlogEditor.jsx`, `UserProfile.jsx` |
| Contest component | Active contests, contest portfolio, contest trades, contest leaderboard. | `contest.js`, `Contest.jsx`, `ContestArena.jsx` |
| Leaderboard component | General ranking by portfolio performance. | `leaderboard.js`, `Leaderboard.jsx` |

| | | |
|---|---|---|
| News component | News retrieval and injection into market context. | `news.js`, `newsInjector.js`, `NewsPanel.jsx` |

## 4.3 Component Diagram

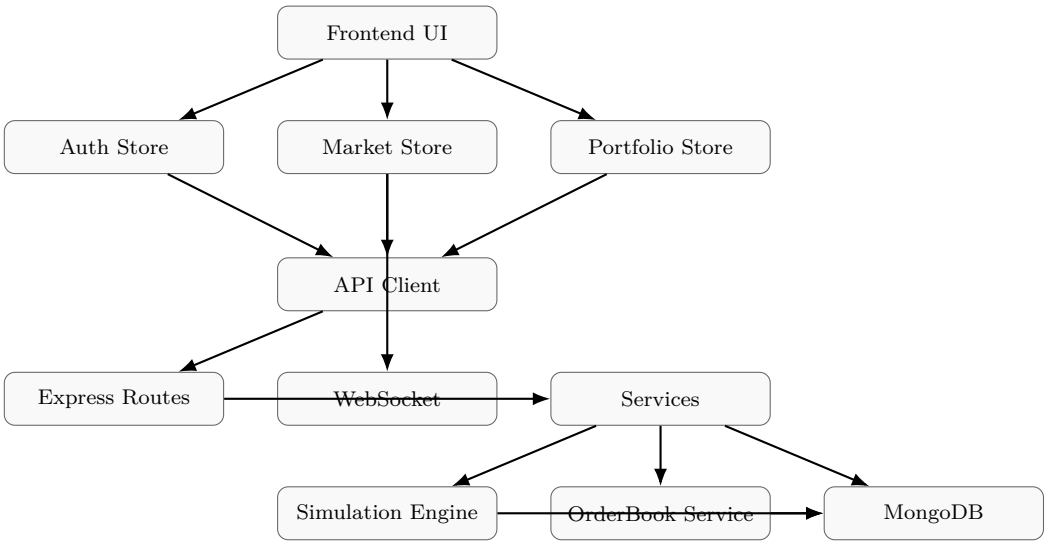


Figure 4.2: Simplified component diagram

## 4.4 Database Design

The database design is document-oriented and implemented with MongoDB. The main collections are:

- `users`: account and profile data;
- `portfolios`: cash, holdings, and portfolio state;
- `transactions`: completed buy/sell transactions;
- `ticks`: simulated price data;
- `leaderboard`: user ranking records;
- `portfolio_snapshots`: historical portfolio value records;
- `contests`: active and historical contests;
- `contest_portfolios`: contest-specific cash and holdings;
- `blog_posts`: draft, published, and archived posts;
- `learning_progress`: lesson and quiz progress;
- `chatbot_history`: stored chatbot conversations.

## 4.5 Detailed Package Design

### 4.5.1 Backend Package Design

```
backend/
  routes/
    auth.js
    market.js
    orders.js
    portfolio.js
    leaderboard.js
    advisor.js
    scenarios.js
    news.js
    contest.js
    blogs.js
    learning.js
    chatbot.js
  services/
    db.js
    simulationEngine.js
    stockData.js
    orderBook.js
    riskMetrics.js
    newsInjector.js
    newsService.js
    blogService.js
    chatbotService.js
  websocket/
    priceStream.js
  server.js
```

### 4.5.2 Frontend Package Design

```
frontend/src/
  pages/
    Landing.jsx
    Simulation.jsx
    Portfolio.jsx
    Leaderboard.jsx
    Contest.jsx
```

```
  ContestArena.jsx
  Blogs.jsx
  BlogDetail.jsx
  BlogEditor.jsx
  Learn.jsx
  UserProfile.jsx
components/
  simulation/
  learn/
  chatbot/
  layout/
  shared/
store/
  authStore.js
  marketStore.js
  portfolioStore.js
  orderStore.js
  learningStore.js
  chatbotStore.js
  contestStore.js
  leaderboardStore.js
lib/
  api.js
  learningData.js
  consultantData.js
  chatbotKnowledge.js
  chatbotSafety.js
  indicators.js
```

## 4.6 API Design

Table 4.2: Main API endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| POST | `/api/auth/signup` | Create user and default portfolio. |
| POST | `/api/auth/signin` | Sign in. |
| GET | `/api/auth/profile` | Get user profile. |
| PUT | `/api/auth/profile` | Update display name. |
| GET | `/api/market/stocks` | Get stocks with current prices. |

| | | |
|---|---|---|
| GET | /api/market/history/:ticker | Get historical price bars. |
| GET | /api/market/ohlcv/:ticker | Get aggregated OHLCV candles. |
| GET | /api/market/quote/:ticker | Get quote. |
| POST | /api/orders | Place simulated order. |
| GET | /api/orders | Get open/filled orders. |
| DELETE | /api/orders/:id | Cancel order. |
| GET | /api/portfolio | Get portfolio. |
| POST | /api/portfolio/trade | Execute buy/sell. |
| GET | /api/portfolio/history | Get portfolio snapshots. |
| GET | /api/portfolio/risk | Get risk metrics. |
| GET | /api/leaderboard | Get leaderboard. |
| GET | /api/contest/active | Get active contests. |
| POST | /api/contest/join | Join contest. |
| GET | /api/contest/portfolio | Get contest portfolio. |
| POST | /api/contest/trade | Trade in contest. |
| GET | /api/contest/leaderboard | Get contest leaderboard. |
| GET | /api/news | Get news. |
| POST | /api/scenarios/:id/activate | Activate market scenario. |
| POST | /api/scenarios/deactivate | Deactivate scenario. |
| POST | /api/advisor/chat | Mock advisor response. |
| POST | /api/advisor/backtest | Mock backtest. |
| GET | /api/blogs | List published posts. |
| GET | /api/blogs/:slug | Get one published post by slug. |
| GET | /api/blogs/me | List the signed-in user's own posts. |
| GET | /api/blogs/profiles/:userId | Get a public profile and that user's published posts. |
| POST | /api/blogs | Create draft post. |
| PUT | /api/blogs/:id | Update own post. |
| PATCH | /api/blogs/:id/publish | Publish own post. |
| PATCH | /api/blogs/:id/archive | Archive own post. |
| DELETE | /api/blogs/:id | Delete own post. |
| POST | /api/blogs/:id/vote | Vote on post. |
| POST | /api/blogs/:id/comments | Comment on post. |
| DELETE | /api/blogs/:id/comments/:commentId | Delete own comment or delete comments on own post. |
| GET | /api/learning/progress | Get learning progress. |
| PUT | /api/learning/progress | Update learning progress. |
| POST | /api/learning/lesson/section | Mark lesson section. |
| POST | /api/learning/quiz/result | Save quiz result. |

| | | |
|---|---|---|
| POST | `/api/chatbot/message` | Send chatbot message. |
| GET | `/api/chatbot/history` | Get chatbot history. |
| DELETE | `/api/chatbot/history` | Clear chatbot history. |

## 4.7 Interface Design

Table 4.3: Main user interfaces

| **Page** | **Main features** |
|---|---|
| Landing | Hero section, feature introduction, stock preview, services and advisor introduction. |
| Simulation | Watchlist, ticker header, stock chart, order form, order book, news panel, scenario selector, time and sales, positions. |
| Portfolio | Cash, total value, holdings, realized/unrealized P/L, allocation, transaction history, risk metrics. |
| Learn | Learning dashboard, lessons, quizzes, badges, market lab, pattern game, recommended lessons. |
| Blogs | Published posts, combined stock/user filter, sorting, animated post cards, votes, comments, and author navigation. |
| BlogEditor | Create and edit blog posts, preview content, and extract stock tags from `$TICKER` mentions. |
| UserProfile | Public profile, user post list, and owner post management for draft, published, and archived posts. |
| Leaderboard | Rank users by portfolio performance. |
| Contest | Active contests and reward information. |
| ContestArena | Contest-specific trading interface and contest portfolio. |
| ChatbotWidget | Floating learning assistant and conversation panel. |

# Chapter 5

# Demo Program Implementation

## 5.1 Tools and Libraries

Table 5.1: Technology stack

| **Layer** | **Technology** |
|---|---|
| Frontend | React, Vite |
| Routing | React Router |
| State management | Zustand |
| Charts | Lightweight Charts, Recharts, custom chart components |
| Backend | Node.js, Express |
| Real-time communication | WebSocket using `ws` |
| Database | MongoDB |
| Security | `bcryptjs` for password hashing |
| Deployment | Docker, Docker Compose, Nginx frontend container |
| Styling | CSS files, reusable components, responsive layout |

## 5.2 Implemented Functions

### 5.2.1 Authentication

The authentication module supports sign-up, sign-in, profile retrieval, and display name update. Passwords are hashed before being stored. When a user account is created, the system also creates a default portfolio so the learner can start trading immediately.

### 5.2.2 Market Simulation

The market simulation is one of the central components of the project. The backend initializes a simulation engine, generates historical bars, starts a real-time tick loop, and broadcasts price updates through WebSocket. The simulation logic includes jump behavior, momentum, mean reversion, volatility clustering, sector effects, market scenarios, and news injection.

### 5.2.3 Trading and Order Management

The system provides two related trading mechanisms. First, direct portfolio trading allows users to buy or sell at the current simulated price. Second, the order system allows order placement, display of open and filled orders, and cancellation. Each successful trade updates cash, holdings, transactions, and leaderboard state.

### 5.2.4 Portfolio Analytics

The portfolio page displays virtual cash, holdings, total portfolio value, profit and loss, allocation, transaction history, and risk metrics. Risk metrics include volatility, Sharpe ratio, and maximum drawdown. These indicators help users connect trading actions with risk and performance outcomes.

### 5.2.5 Learning Center

The learning center includes lessons, quizzes, learning paths, badges, practice tasks, market analysis lab, and pattern game. The backend stores learning progress and quiz results, while the frontend presents structured educational activities.

### 5.2.6 Chatbot and Advisor

The chatbot widget allows users to ask questions about the platform, trading concepts, portfolio risk, order types, lessons, and simulated market behavior. The chatbot applies safety rules to avoid real financial advice and may provide suggestion cards to guide the user to relevant features. The advisor route provides mock educational advisory responses and backtest results.

### 5.2.7 Blog and Community

The blog module was implemented as a community discussion feature for the simulated stock learning platform. It allows signed-in users to create draft posts, update their own posts, publish posts, archive posts, delete posts, vote on posts, and comment

on posts. Public readers can browse published posts, while ownership rules protect author-only actions.

On the backend, the blog feature is implemented through `backend/routes/blogs.js` and `backend/services/blogService.js`. The route file exposes endpoints for listing posts, creating drafts, updating posts, publishing, archiving, deleting, voting, commenting, deleting comments, and viewing public user profiles. The service file contains the main business rules: post validation, slug generation, owner authorization, vote counting, comment permission checks, public author-name hydration, sorting, filtering, and MongoDB serialization. Blog data is stored in the `blog_posts` collection with fields for title, slug, excerpt, content, author, status, extracted stock tags, votes, comments, timestamps, and publication state.

On the frontend, the main pages are `Blogs.jsx`, `BlogDetail.jsx`, `BlogEditor.jsx`, and `UserProfile.jsx`. The Blogs page displays published posts using clickable cards, entrance animation, sorting by time/upvotes/comments/oldest, and a combined filter that can search by stock ticker or username. Each card displays author information, stock tags, upvote count, comment count, excerpt preview, and an owner-only edit action. The Blog Detail page displays the full post, voting controls, comments, comment deletion for permitted users, and an owner-only edit button. The Blog Editor supports draft creation and editing. Instead of a separate stock-tag field, stock tags are extracted automatically from `$TICKER`-style mentions in the title, excerpt, or content. The preview panel shows detected stock tags before publishing.

The profile experience was integrated with blog management. Public profile pages list a user's published posts and allow sorting by newest, upvotes, comments, or oldest. When the signed-in user opens their own profile, the page also acts as the post management page: it lists draft, published, and archived posts; clicking a row opens the editor; and the user can edit, archive, or delete posts according to their status. This avoids a separate management tab and keeps blog ownership actions under the Profile page.

Several usability improvements were added to make the blog tab more interactive. Blog cards rise gradually from the bottom when the Blogs tab is opened, with reduced-motion support for accessibility. The whole blog card is clickable, but vote, comment, stock tag, author, and edit controls preserve their own behavior. Stock mentions such as `$SCT` are highlighted in the blog viewer and link back to the Blogs page filtered by that stock. Author names are clickable and lead to the author's public profile. If the post belongs to the current user, the author label is displayed as "By you".

The comment system supports adding comments on published posts and deleting comments. A user can delete their own comment, and a post author can delete comments under their own posts. This permission rule is enforced on the backend, not only in the frontend interface. The voting system allows upvotes and downvotes, but the

user-facing vote count displays only the number of upvotes. Sorting by "Top Rated" was changed to use raw upvote count.

### 5.2.8 Contest and Leaderboard

The contest module displays active contests, allows users to join contests, initializes contest-specific portfolios, restricts contest trades to allowed tickers, executes contest trades, and displays contest leaderboards. The general leaderboard ranks users according to simulation performance.

## 5.3 Demo Interface Screenshots

The following screenshots should be inserted after running the final demo:

- Figure 5.1: Landing page.
- Figure 5.2: Sign-up and sign-in modal.
- Figure 5.3: Simulation dashboard with stock chart and order form.
- Figure 5.4: Portfolio dashboard with holdings and risk metrics.
- Figure 5.5: Learning center and quiz module.
- Figure 5.6: Chatbot widget and response panel.
- Figure 5.7: Blog list, blog editor, blog detail, and user profile management.
- Figure 5.8: Contest page and contest arena.
- Figure 5.9: Leaderboard page.

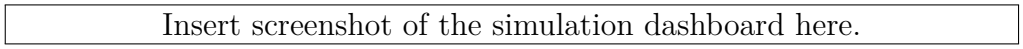


Figure 5.1: Placeholder for simulation dashboard screenshot

# Chapter 6

# System Testing

## 6.1 Testing Strategy

The testing strategy follows the V-model logic: each requirement should be verified by one or more test cases. The system is tested at multiple levels: unit testing for individual services, integration testing for routes and database operations, functional testing for user workflows, non-functional testing for quality attributes, and user acceptance testing for the final demo.

## 6.2 Functional Test Cases

Table 6.1: Functional test cases

| ID | Test case | Input | Expected output |
|---|---|---|---|
| TC-01 | Sign up with valid information | Email, password with at least six characters, display name | User is created and default portfolio is initialized. |
| TC-02 | Sign up with existing email | Existing email | Error message is returned. |
| TC-03 | Sign in with wrong password | Valid email, wrong password | Authentication fails. |
| TC-04 | View stock list | Open simulation page | Stocks and current prices are displayed. |
| TC-05 | Receive WebSocket tick | Backend and WebSocket connected | Prices update in the UI without page refresh. |
| TC-06 | WebSocket disconnected | Stop or interrupt WebSocket | Frontend recovers or falls back gracefully. |

| | | | |
|---|---|---|---|
| TC-07 | Buy with enough cash | Buy 10 shares of a stock | Cash decreases, holdings increase, transaction is recorded. |
| TC-08 | Buy with insufficient cash | Buy more shares than cash allows | Trade is rejected. |
| TC-09 | Sell with enough shares | Sell existing holding | Cash increases, holdings decrease. |
| TC-10 | Sell without enough shares | Sell more than current holding | Trade is rejected. |
| TC-11 | Place order | Valid order data | Order is created and listed. |
| TC-12 | Cancel order | Existing order ID | Order is cancelled. |
| TC-13 | View portfolio | Logged-in user | Portfolio value, holdings, profit/loss, and transactions are displayed. |
| TC-14 | Get risk metrics | Portfolio snapshots exist | Sharpe ratio, volatility, and maximum drawdown are returned. |
| TC-15 | Join contest | User ID and active contest ID | Contest portfolio is initialized. |
| TC-16 | Trade disallowed contest ticker | Ticker not allowed in contest | Trade is rejected. |
| TC-17 | Complete quiz | Submit quiz answers | Score and result are saved. |
| TC-18 | Publish blog post | Draft post | Post becomes public. |
| TC-19 | Comment on blog | Comment content | Comment is added to the post. |
| TC-20 | Delete blog comment | Comment owner or post owner deletes a comment | Comment is removed and comment count is updated. |
| TC-21 | Vote on blog post | Upvote or downvote request | Vote state is updated and upvote count is displayed. |
| TC-22 | Filter blogs | Stock mention or username query | Blog list returns posts matching the stock tag or author. |
| TC-23 | Extract stock mention | Post content containing `$SCT` | Stock tag is extracted and highlighted in the viewer. |

| | | | |
|---|---|---|---|
| TC-24 | Manage own profile posts | Open own profile | Draft, published, and archived posts are listed with edit/archive/delete actions. |
| TC-25 | Ask chatbot | User message | Educational response and disclaimer are returned when necessary. |

## 6.3 Non-functional Testing

Table 6.2: Non-functional testing plan

| **Requirement** | **Test method** |
|---|---|
| Password security | Inspect user records and confirm that raw passwords are not stored. |
| Data integrity | Execute repeated buy/sell actions and confirm that cash and shares never become negative. |
| Real-time behavior | Observe whether the simulation page updates prices through WebSocket without refreshing. |
| Fallback reliability | Disconnect WebSocket and confirm the UI remains usable or recovers gracefully. |
| Maintainability | Check whether routes, services, stores, and components are separated by domain. |
| Usability | Confirm that users can navigate to Simulation, Portfolio, Learn, Blogs, Contest, and Leaderboard from the main navigation. |
| Ethical safety | Ask the chatbot for real investment advice and confirm that it avoids direct buy/sell recommendations and provides an educational disclaimer. |

## 6.4 Requirement Traceability Matrix

Table 6.3: Requirement traceability matrix

| **Requirement ID** | **Requirement** | **Component** | **Test case** |
|---|---|---|---|

| | | | |
|---|---|---|---|
| FR-AUTH-1 | User can sign up. | Auth route, AuthStore | TC-01 |
| FR-MKT-1 | User can view stock prices. | Market route, Simulation page | TC-04 |
| FR-MKT-3 | System streams live prices. | SimulationEngine, WebSocket | TC-05 |
| FR-TRD-1 | User can buy stock. | Portfolio route, OrderForm | TC-07 |
| FR-TRD-2 | System rejects insufficient cash. | Portfolio route | TC-08 |
| FR-TRD-3 | User can sell stock. | Portfolio route | TC-09 |
| FR-PRT-5 | System calculates risk metrics. | RiskMetrics service | TC-14 |
| FR-LRN-3 | User can submit quiz. | Learning route, QuizModule | TC-17 |
| FR-SOC-1 | User can ask chatbot. | Chatbot route, ChatbotWidget | TC-25 |
| FR-SOC-4 | User can publish blog. | Blog route, BlogEditor | TC-18 |
| FR-SOC-5 | User can vote, comment, filter, and manage blog posts. | Blog route, BlogService, Blogs, BlogDetail, UserProfile | TC-19–TC-24 |
| FR-SOC-6 | User can join contest. | Contest route, Contest page | TC-15 |

# Chapter 7

# Installation and User Guide

## 7.1 System Requirements

### 7.1.1 Hardware Requirements

- A laptop or desktop computer capable of running Node.js and a modern browser.
- At least 4 GB RAM is recommended for local development.
- Internet connection is required if external APIs or remote MongoDB are used.

### 7.1.2 Software Requirements

- Node.js and npm.
- Docker and Docker Compose for containerized deployment.
- MongoDB connection string.
- Modern web browser such as Chrome, Edge, or Firefox.
- Optional API keys such as `GNEWS_API_KEY` and `GEMINI_API_KEY`.

## 7.2 Docker Installation

The easiest way to run the complete stack is Docker Compose.

```
docker compose up -d --build
```

After the containers start, open:

```
http://localhost
```

To stop the application:

```
docker compose down
```

## 7.3 Manual Backend Setup

```
cd backend
npm install
npm start
```

A typical backend environment configuration is:

```
MONGODB_URI=mongodb+srv://<username>:<password>@<cluster>/<database>?
    retryWrites=true&w=majority
GNEWS_API_KEY=<optional>
GEMINI_API_KEY=<optional>
```

## 7.4 Manual Frontend Setup

```
cd frontend
npm install
npm run dev
```

The Vite development server usually starts on port 5173 or 5174.

## 7.5 User Guide

### 7.5.1 For New Users

1. Open the landing page.
2. Sign up or sign in.
3. Go to the Simulation page.
4. Select a stock from the watchlist.
5. Place a buy or sell order.
6. Open the Portfolio page to review cash, holdings, profit/loss, and risk metrics.
7. Open the Learn page to study lessons and complete quizzes.
8. Ask the chatbot for explanations about platform features or financial concepts.

### 7.5.2 For Contest Users

1. Open the Contest page.
2. Select an active contest.
3. Join the contest.
4. Open the Contest Arena.
5. Trade allowed tickers using the contest portfolio.

6. Check the contest leaderboard.

### 7.5.3 For Blog Users

1. Open the Blogs page.
2. Browse published posts, sort them by newest, upvotes, comments, or oldest, and use the filter box to search by stock ticker or username.
3. Click *Write a Post* to create a draft post.
4. Write the title, excerpt, and content. To tag a stock, type a mention such as `$SCT` or `$HEAL` directly inside the post.
5. Save the draft and publish the post when it is ready.
6. Read published posts, open author profiles, upvote or downvote posts, and add comments.
7. Delete your own comments. If you are the post author, you may also delete comments under your own post.
8. Open your Profile page to manage your own posts. From there, edit drafts or published posts, archive published posts, and delete draft or archived posts.

# Chapter 8

# Project Management and Configuration Management

## 8.1 Development Methodology

The project follows an Agile-inspired iterative development approach. This approach is suitable because the system contains multiple relatively independent modules such as authentication, simulation, trading, portfolio, learning, chatbot, blog, contest, and leaderboard. Each module can be designed, implemented, tested, and improved incrementally.

Table 8.1: Suggested sprint plan

| **Sprint** | **Main deliverables** |
|---|---|
| Sprint 1 | Requirement analysis, UI prototype, architecture design. |
| Sprint 2 | Authentication, MongoDB connection, user portfolio initialization. |
| Sprint 3 | Simulation engine, stock data, historical prices, WebSocket stream. |
| Sprint 4 | Trading, order book, portfolio dashboard. |
| Sprint 5 | Learning center, quizzes, badges, progress tracking. |
| Sprint 6 | Chatbot, advisor, blog/community features. |
| Sprint 7 | Contest, leaderboard, testing, Docker deployment, final report. |

## 8.2 Work Breakdown Structure

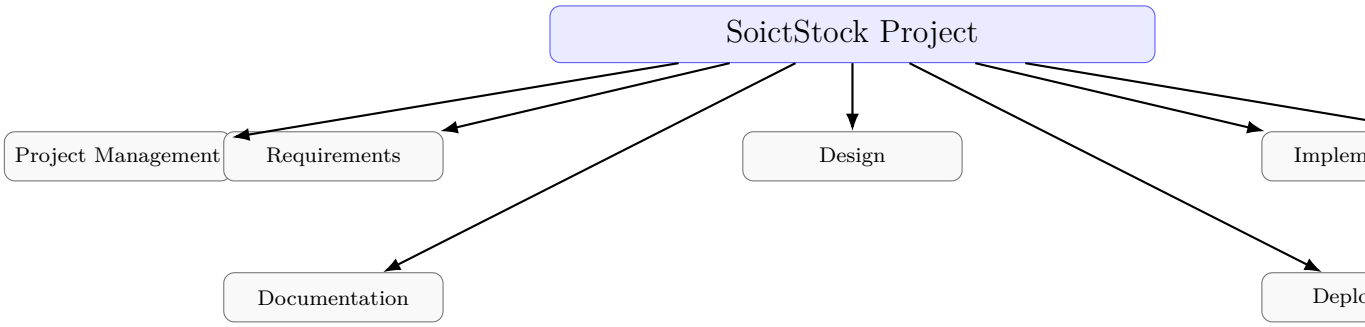


Figure 8.1: Work breakdown structure

## 8.3 Risk Management

Table 8.2: Risk assessment and mitigation

| **Risk** | **Impact** | **Mitigation** |
|---|---|---|
| Simulation engine is too complex for a course demo. | High | Explain the algorithm clearly and test with selected tickers and scenarios. |
| WebSocket connection fails. | Medium | Provide graceful recovery and frontend fallback behavior. |
| Portfolio inconsistency after trading. | High | Validate cash and holdings before every trade and test repeated transactions. |
| Chatbot gives real investment advice. | High | Apply chatbot safety rules and educational disclaimers. |
| Blog content may be misleading. | Medium | Add community and educational disclaimers and future moderation. |
| Scope is too large. | High | Prioritize simulation, trading, portfolio, learning, chatbot, and core community features. |
| MongoDB connection issue. | High | Provide `.env.example`, Docker setup, and manual setup guide. |

## 8.4 Configuration Management

Software configuration management is necessary because the project contains source code, documentation, configuration files, seed data, and deployment artifacts that

change throughout development.

### 8.4.1 Configuration Items

- Backend source code.
- Frontend source code.
- MongoDB collection design and seed data.
- Stock data and learning content data.
- Chatbot knowledge and safety rules.
- Dockerfile and `docker-compose.yml`.
- API documentation.
- Test cases and test results.
- Report, diagrams, screenshots, and user guide.

### 8.4.2 Git Strategy

A recommended Git strategy is:

- `main`: stable integrated version.
- `develop`: integration branch for tested features.
- `feature/auth`: authentication and profile.
- `feature/simulation`: market simulation and WebSocket.
- `feature/trading`: orders, portfolio, and transactions.
- `feature/learning`: lessons, quizzes, badges.
- `feature/chatbot-blog`: chatbot and community content.
- `feature/contest`: contest and leaderboard.

### 8.4.3 Change Management Workflow

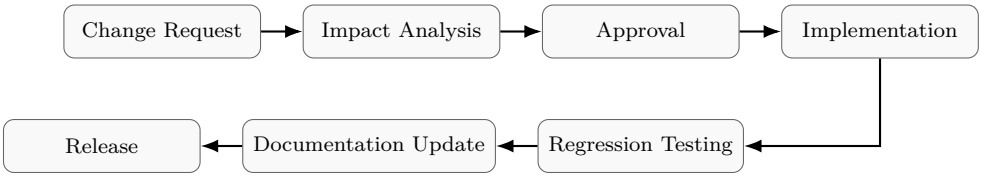


Figure 8.2: Change management workflow

# Chapter 9

# Evaluation and Future Work

## 9.1 System Evaluation

### 9.1.1 Strengths

- The project provides a full-stack implementation with a React/Vite frontend, Express backend, MongoDB persistence, and WebSocket streaming.
- The market simulation is more realistic than a simple random walk because it includes jump behavior, momentum, mean reversion, volatility clustering, market scenarios, and news effects.
- Trading and portfolio modules are connected, so user actions immediately influence portfolio value and rankings.
- The learning center is integrated with quizzes, badges, market analysis lab, and practice activities.
- The chatbot supports platform guidance and financial education while including safety disclaimers.
- Blog/community and contest/leaderboard modules increase engagement.
- Docker setup improves deployment reproducibility.

### 9.1.2 Limitations

- Authentication is not yet a complete production-grade JWT/session authorization system.
- There is no complete role-based admin dashboard.
- There is no real consultant booking system.
- Advisor and backtest functions are mock or simulation-only.
- The simulated market does not fully represent a real stock exchange.
- Some data and configuration are still demo-oriented.
- External news or AI behavior may depend on optional API keys.

- Contest rewards are simulated.

## 9.2 Future Work

Future development can focus on:

- adding a role-based admin dashboard;
- improving authentication with JWT and authorization middleware;
- adding a real consultant appointment booking module;
- extending portfolio analytics and risk explanation;
- adding real historical datasets for backtesting mode;
- adding collaborative trading competitions;
- adding notification and alert systems;
- adding moderation tools for community blogs;
- improving mobile responsiveness and accessibility;
- explaining how each pricing factor affects the simulated price in a transparent way.

# Chapter 10

# Conclusion

This report presented the software engineering design and implementation plan for SoictStock, a virtual stock market simulation and financial education platform. The project addresses the need for a safe, interactive, and educational environment where beginners can learn stock market concepts through practice without risking real money.

The system combines several major functions: real-time market simulation, virtual trading, order management, portfolio analytics, risk metrics, learning paths, quizzes, chatbot guidance, blogs, contests, and leaderboards. Based on the implemented source code, the project follows a modular client–server architecture with a React/Vite frontend, an Express backend, MongoDB storage, and WebSocket price streaming.

From a software engineering perspective, the project applies requirement analysis, functional and non-functional requirements, use case modeling, class and data modeling, architecture design, interface design, implementation planning, testing, project management, and configuration management. The current version is a meaningful MVP for educational stock market simulation, while future work can improve authentication, administration, consultant booking, data realism, and moderation.

Overall, SoictStock demonstrates how software engineering methods can transform a complex educational idea into a structured, testable, maintainable, and extensible software system.

# Appendix A

# API Details

This appendix can be expanded with request and response examples for the main APIs. Example:

```
POST /api/portfolio/trade
Request body:
{
  "userId": "...",
  "ticker": "ABC",
  "type": "buy",
  "quantity": 10
}

Expected response:
{
  "success": true,
  "transaction": { ... },
  "portfolio": { ... }
}
```

# Appendix B

# Sample Data

Recommended sample data for the demo:

- At least 8–12 simulated stocks from different sectors.
- Several market scenarios such as financial crisis, technology bubble, pandemic shock, and inflation.
- Learning lessons covering stock basics, order types, portfolio diversification, risk management, and technical patterns.
- Quiz questions for each learning path.
- Sample blog posts and comments.
- At least one active contest.

# Appendix C

# Full Test Cases

The testing chapter includes the main functional and non-functional test cases. This appendix can be used to store detailed screenshots, logs, and step-by-step testing evidence.

# Appendix D

# Screenshots

Insert full-size screenshots of the final demo here. Recommended screenshots:

1. Landing page.
2. Authentication modal.
3. Simulation dashboard.
4. Portfolio dashboard.
5. Learning center.
6. Chatbot panel.
7. Blog page and editor.
8. Contest page and contest arena.
9. Leaderboard page.

# Bibliography

[1] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, latest edition.

[2] IT3180E Course Materials. *Introduction to Software Engineering: SDLC, Agile, Project Management, SCM, Requirements, Design, Construction, and Quality Assurance*. Hanoi University of Science and Technology.

[3] SoictStock source code package. *React/Vite frontend, Express backend, MongoDB persistence, WebSocket price streaming, market simulation, trading, learning, chatbot, blog, contest, and leaderboard modules*.